

- Settlement Reconciliation — Automation Proposal
 - 1. Executive Summary
 - 2. Current Process (As-Is) — Reference: YES Bank UPI
 - 2.1 Inputs (4 reports collected daily)
 - 2.2 Manual Reconciliation Workflow
 - 2.3 The Five Scenarios (from YES Bank SOP)
 - 2.4 Multiply by 7 PG partners × multiple modes
 - 3. Pain Points (Why Automate Now)
 - 4. Proposed Solution
 - 4.0 Design Principles (load-bearing)
 - 4.1 Architecture (conceptual)
 - 4.2 What Finance Ops Sees (new daily workflow)
 - 4.3 What stays the same
 - 4.4 What changes for the team
 - 4.5 LLM Governance (where AI is and isn't used)
 - 5. Phased Roadmap
 - 5.1 Out of Scope (Now — for separate proposals)
 - 6. Resources & Dependencies
 - 6.1 Engineering
 - 6.2 Cross-Functional
 - 6.3 Infrastructure
 - 6.4 No Dependencies On
 - 7. Risks & Mitigations
 - 8. Success Metrics
 - 8.1 Pilot (YES Bank UPI) — go/no-go criteria for next PG
 - 8.2 Steady State (all 7 PGs)
 - 8.3 Stretch (post-rollout)
 - 9. Cost / Benefit Summary
 - 10. Decision Required
 - 11. Open Questions for Reviewer
 - 12. Appendices
 - A. SOPs reviewed
 - B. Sample data verified
 - C. Engineering details (for technical reviewers)

Settlement Reconciliation — Automation

Proposal

Status: Draft for review **Author:** Ashish S., AI Engineering **Reviewers:** Engineering Manager · Sr PM · Finance Operations Lead · Product Support Lead **Date:** May 2026 **Pilot target:** YES Bank UPI settlement reconciliation **Estimated timeline:** 8 weeks to first PG live · ~16–20 weeks to all 7 PGs

1. Executive Summary

Our finance team manually reconciles ~1,000+ transactions per PG per day across four separate Excel reports per partner, using VLOOKUP and manual scenario judgement. With **7+ PG partners** (YES, NPST, Paynext, Atom, TPSL, HDFC, RBL) and **multiple transaction modes** (UPI, CC, DC, Netbanking), this is becoming the team's largest daily time sink and a known source of settlement errors, missed refunds, and delayed escalations.

This proposal automates that work using a **multi-leg reconciliation engine** (linear cascade: PG TXN → PG SETTLEMENT → Internal MIS → Bank Statement), already partially built in our existing AI Agents service. The engine produces an audited, exception-only review queue. Finance ops shifts from "reconcile 1,000 rows" to "review the 50–100 exceptions the system flagged."

The downstream Admin DB pipeline (settlement upload → cron → maker-checker → 4 PM auto-cron) is unchanged. This proposal targets only the *reconciliation* step that today happens in Excel.

Metric	Today	After Pilot (YES Bank UPI)
Daily manual reco time per PG	~1–2 hours	~10–20 min (exception review only)
Auto-match rate	n/a (100% manual VLOOKUP)	Target ≥ 80% on launch
Audit trail	Excel overwrites	Every decision logged, reproducible
Late-Authorized / refund cases missed	Periodic	Detected by rule, surfaced for review

Metric	Today	After Pilot (YES Bank UPI)
Scaling to next PG	Onboard a new analyst	Add adapter + rules file in ~1 week

2. Current Process (As-Is) – Reference: YES Bank UPI

This is what the finance team does **every working day** for YES Bank UPI. The shape is similar for the other 6 PG partners with partner-specific column names, file formats, and edge cases.

2.1 Inputs (4 reports collected daily)

#	Report	Source	Cadence	Format
1	MIS Report (PG authority data)	YES Bank MIS Portal (manual download)	Daily morning	XLSX/CSV — ~1,000 rows x 41 columns
2	PG Incoming Report (our internal record)	Metabase (manual download)	Daily	XLSX — ~10–20 rows x 20 columns
3	Consolidated Report (already-settled list)	Metabase	Daily	XLSX — small (< 10 rows)
4	Bank Statement	Email to settlement.refunds@ mailbox OR YES Corporate Portal	Daily	XLSX — ~10–20 batched settlement credits

NPST follows a similar pattern with two settlement files per day (split at 10 AM cutoff) plus an NPST Cosmos Portal transaction report.

2.2 Manual Reconciliation Workflow

Step 1 → Download all 4 reports.

Step 2 → Open the MIS workbook. For each row (≈1,000):

- VL00KUP into PG Incoming sheet by Customer Ref No.
→ Stamp "PG incoming" column (✓ / #N/A)
- VL00KUP into Consolidated sheet
→ Stamp "Consolidated" column (✓ / #N/A – already settled)
- VL00KUP into Bank Statement by BANKREFERENCENUMBER
→ Stamp "Bank" column (✓ / #N/A – cash received)

Step 3 → Classify each MIS row into one of 5 scenarios (see below).

Step 4 → Filter to "settle" rows → prepare settlement CSV in YES MIS format.

Step 5 → Upload CSV → Settlement Cron → Maker-Checker → wait for 4 PM auto-cron.

2.3 The Five Scenarios (from YES Bank SOP)

#	Trigger	Action
1	Success in MIS + Success in PG + funds in bank + NOT in Consolidated	Settle (include in CSV)
2	Timeout in MIS + Failed/Timeout in PG + funds in bank	Refund to source
3	Success in MIS + Late Authorized in PG + funds in bank	Refund to source
4	Failed in both + no funds	Ignore
5	Success in MIS but missing from PG	Escalate to Product Support

NPST has a similar 3-case classification driven by NPCI response code "00" = success.

2.4 Multiply by 7 PG partners × multiple modes

Partner	Modes	Daily volume (est.)	Current daily time
YES Bank	UPI	~1,000 txns	~1–2 hrs
NPST	UPI	~1,500 txns	~1.5–2.5 hrs (2 files per day)
Paynext	UPI/CC/DC/NB	??	??
Atom	UPI/CC/DC/NB	??	??
TPSL	UPI/CC/DC/NB	??	??

Partner	Modes	Daily volume (est.)	Current daily time
HDFC	UPI/CC/DC/NB	??	??
RBL	UPI/CC/DC/NB	??	??

Volumes / times marked ?? need confirmation from the finance team. Initial estimate: total ~7–14 person-hours/day across all partners and modes.

3. Pain Points (Why Automate Now)

#	Pain Point	Today's Impact
1	Manual VLOOKUP across thousands of rows daily	High operator time + ergonomic strain
2	Scenario-classification done by judgement per row	Late-Authorized & duplicate-charge cases are easily missed
3	No SLA visibility ("are we waiting on YES Bank file?")	Knowledge lives in one analyst's head
4	No audit trail — Excel changes overwrite history	Cannot trace why a settlement happened a specific way
5	Each new PG = onboard a new analyst	Doesn't scale linearly with business growth
6	Cannot proactively detect fraud / outage patterns	Reactive only
7	Bus-factor of 1 in some PG-specific edge knowledge	Single-person dependency, holiday/leave risk

4. Proposed Solution

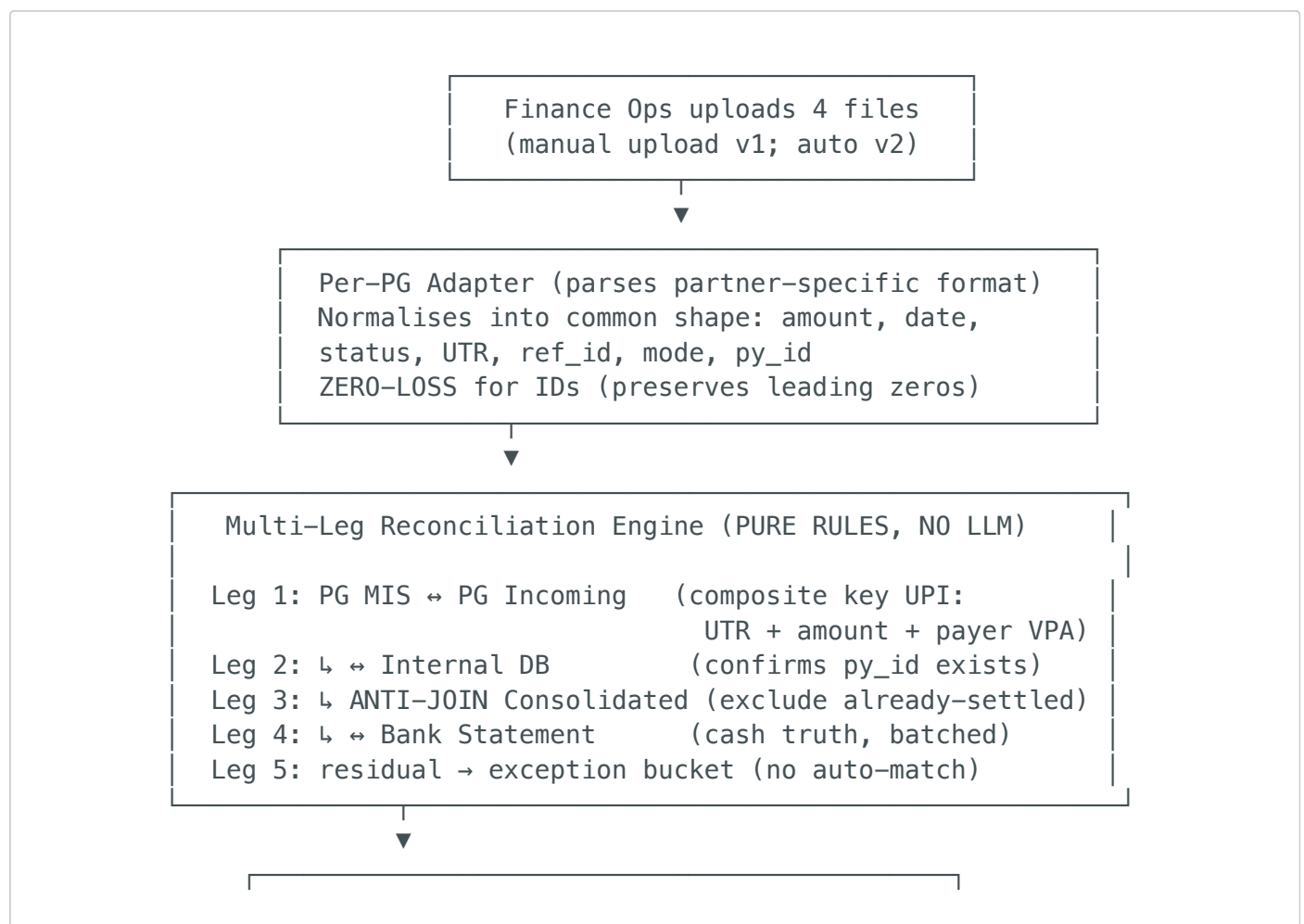
4.0 Design Principles (load-bearing)

Four principles drive every later choice. They came out of design review with the team.

1. **Deterministic-first.** Every reconciliation match and disposition decision is pure-rule code. Same inputs → exactly same outputs, every run. No LLM in the decision path. Anything the rules cannot match goes to the exception report — not "the AI's best guess".
2. **LLM is governed, never authoritative.** AI is used in three narrow read-only places (see §4.5). It cannot create, modify, or override a reconciliation decision. Every LLM call is audit-logged.
3. **Reports first, UI second.** Finance team works in Excel. We generate the spreadsheets they would have produced manually — per-leg CSVs, settlement CSV, exception XLSX. The dashboard is a thin operational shell around those downloads.
4. **Every decision is reproducible and citable.** Each row in every output report carries `legId`, `ruleId`, `joinKeyUsed`, and `normalizations[]`. An auditor can ask "why did this settle?" and we point to the exact rule and field values that fired.

These principles distinguish us from vendors like HighRadius (which uses AI to auto-clear up to 90% — including some fuzzy matches that we'd push to the exception queue). We accept a lower auto-match rate (~80% target from pure rules) in exchange for **100% auditability**. The right call for Indian payments settlement.

4.1 Architecture (conceptual)



Disposition Engine (PURE-RULE TypeScript)
SOP scenarios encoded as typed functions
→ settle / refund / ignore / escalate /
exception (anything not matched)
NO LLM. Same input → same output.



REPORT PACK (the v1 ship gate – downloaded by Ops)

- | 00_run_summary.csv (per-leg counts)
- | 01..04_leg<N>/{matched,unmatched}.csv
- | 05_dispositions.csv (every row + ruleId + reason)
- | 06_settlement_upload.csv ← into Admin DB
- | 07_exception_report.xlsx ← finance team works
- | 08_audit_log.csv (every match decision)
- | 09_warnings.csv (zero-pad, schema drift, etc.)



(thin operational layer)

OpenArc Dashboard (file readiness + run status +
prominent download buttons – NO decisions table)



Settlement CSV → existing Admin DB
Settlement Cron → Maker-Checker
→ 4 PM Auto-Cron (UNCHANGED)

4.2 What Finance Ops Sees (new daily workflow)

Morning: → Open OpenArc Reconciliation dashboard
→ "File readiness" widget shows which of 4 YES files have arrived
→ Upload missing files (drag-and-drop)
→ Click "Run reconciliation"
→ System runs in ≤ 5 minutes (deterministic; no LLM in critical path)
→ Click "Download Report Pack" – gets a ZIP of CSVs + the exception XLSX
→ Open exception_report.xlsx in Excel
→ Review the ~50–100 exceptions; finance team applies their existing judgement
→ Open settlement_upload.csv → upload to Admin DB (unchanged downstream)

Time per partner per day: from ~1–2 hours → ~10–20 minutes. Finance team's effort shifts to where it adds the most value — exception judgement in Excel — and away from

mechanical VLOOKUP. **They keep their Excel-native workflow; we just produce the right spreadsheets for them.**

4.3 What stays the same

- The 5-scenario classification logic (encoded as rules per partner)
- The downstream Admin DB pipeline (settlement upload → cron → maker-checker → 4 PM auto-cron)
- Maker-checker approval flow
- Existing reporting / audit trails outside our system

4.4 What changes for the team

Function	Today	After
Finance Recon Analyst	Manual VLOOKUP + scenario classification + CSV preparation	Download report pack; review <code>exception_report.xlsx</code> ; upload <code>settlement_upload.csv</code>
Product Support	Receives ad-hoc escalations via email/Slack	Receives structured escalation rows in <code>exception_report.xlsx</code> with <code>ruleId</code> + <code>reasonText</code>
NPST Team / PG Partner Teams	Email back-and-forth on discrepancies	Same — escalation channel unchanged
Settlement Checker	Maker-checker approval	Same — unchanged

4.5 LLM Governance (where AI is and isn't used)

Auditable money work cannot be delegated to a non-deterministic model. We use AI deliberately and narrowly. Every LLM call is logged with model, prompt, and response so we can defend any decision to an auditor.

Use case	What LLM does	Decision authority	Frequency	Used in v1?
Reconciliation match decision	—	NONE — pure deterministic rules	—	No. Settlement runs with llm: 'off' flag.
Disposition (settle/refund/ignore/escalate)	—	NONE — pure deterministic rules encoding SOP scenarios	—	No. Same flag.
Onboarding a new PG	Suggests how new file's columns map to canonical identifier classes (e.g. "this column is the UTR, that one is the merchant ref")	Suggestion only. Suggestions persist as status: 'proposed' rules; operator must explicitly activate before they influence anything.	Run once per new PG	Yes (during Phase 4)
Exception summary text	For each exception row, writes one plain-English sentence ("₹2,499	None — decoration on top of the deterministic decision. Cannot change bucket, data, or recommendation.	Per-run, per exception row	Yes (Phase 4, optional)

Use case	What LLM does	Decision authority	Frequency	Used in v1?
	credit appears in bank but no MIS row for May 21")			
Pattern observation alerts	"8 Late-Authorized refunds in last hour from same merchant — possible outage?"	None — informational signal posted to Slack/dashboard	Streaming background	No (Phase 5+ if useful)

Audit answer to "why did this settle?" is always *"deterministic rule X fired because field A == y and field B == z"* — never *"the LLM decided"*. The fuzzy-match agent and LLM disposition agent we built earlier stay in the codebase but are gated off for settlement; they may be appropriate for other future workflows where wrong-match risk is lower.

Rule provenance tracking (borrowed from EndClose's pattern): every rule and column mapping carries `source: 'user' | 'ai' | 'default'` and `status: 'proposed' | 'active'`. AI-authored anything starts as `proposed` — only operator activation can promote it. Dashboard surfaces "5 proposed AI rules awaiting review" as a top banner. Combined with the per-LLM-call audit log, we can defend *every* rule's origin to compliance.

5. Phased Roadmap

All durations are engineering estimates assuming 1 full-time backend engineer with design support from our existing AI Agents team. Estimates exclude finance team validation time and any infra procurement.

Phase	Duration	Scope	Acceptance Criteria
0 — Discovery	1 wk	Sit with finance team. Confirm file cadence, edge cases, SLA expectations, success criteria. Sample masked file already collected (YES Bank UPI ✓).	Signed-off acceptance criteria + finance team committed to 1 wk pilot validation
1 — Framework foundation	2 wks	Extend reconciliation engine to support multi-leg cascades, composite join keys (UTR + amount + payer_vpa), and anti-join (exclude-if-present). Backwards-compatible with current 5 reco configs.	All existing reco runs produce identical output (snapshot diff).
2 — YES Bank UPI pilot	2 wks	Build YES Bank adapter for all 4 input files. Author config with 4 legs. Encode the 5 scenarios as disposition rules.	Side-by-side against finance team's manual reco for 5 real days: ≥ 80% auto-match, zero false-settle, zero missed refunds.
3 — UI extensions	1 wk	Per-leg progress timeline, file-readiness widget, workflow-type tab, Excel export matching YES MIS format.	Finance team can complete an end-to-end day in the dashboard with no spreadsheet fallback.
4 — NPST rollout	2 wks	Build NPST adapters (2 settlement files + Cosmos report). Author config + rules. NPCI-code-based status normalisation.	Same acceptance bar as YES. Validates the framework scales to a second PG.
5 — Remaining 5 PGs	~10 wks	One PG per ~2 weeks: Paynext → Atom → TPSL → HDFC → RBL. Each = new adapters + config + rules.	Each PG passes the 5-day side-by-side test before going live.

Cumulative timeline:

- **Week 8:** YES Bank UPI live + dashboard ready (Phases 0–3)
- **Week 10:** NPST live (Phase 4)
- **Week 20:** All 7 PGs live (Phase 5)

5.1 Out of Scope (Now — for separate proposals)

Item	When	Why deferred
Other modes (CC, DC, Netbanking)	Phase 6+	UPI is highest volume; mode dimension is purely an adapter swap once the framework is proven
Refunds workflow (separate finance process)	Q3 2026	Distinct daily process; uses same primitives but different rules
Disputes workflow (multi-day state machine)	Q4 2026	Needs different persistence model — bigger separate project
Exceptions report builder	Q3 2026	Read-only view over reco data; cheap to add once decisions are structured
Automated file intake (SMTP / portal scraping)	After pilot stable	Manual upload is sufficient for v1; automation is incremental

6. Resources & Dependencies

6.1 Engineering

- **1 backend engineer** (full-time, 8 weeks for Phases 1–4; part-time after)
- **Existing AI Agents team** (design review, code review, no incremental headcount)
- **OpenArc dashboard team** — light touch (~1 week aggregate for UI extensions)

6.2 Cross-Functional

- **Finance Operations** (1 ops analyst as design partner) — daily access during Phase 2 (~2 wks); weekly during Phase 4–5

- **Product Support Lead** — confirm escalation routing & escalation card format
- **NPST relationship owner** — confirm 2-file-per-day SLA + escalation channel
- **Admin DB owner** — verify settlement CSV format matches current upload expectations (likely no change)

6.3 Infrastructure

- No new infrastructure cost. Lives in our existing OpenArc AI Agents service (**my-test-agent**)
- Storage: reco decisions stored locally (LibSQL today; migration to Postgres in OpenArc's existing reco_decisions table already done)
- No new API credentials required for the pilot (manual upload). PG portal credentials will be needed when we automate intake in Phase 6+

6.4 No Dependencies On

- **✗** Admin DB changes (downstream pipeline is unchanged)
- **✗** New vendor / SaaS procurement
- **✗** PG partner cooperation (uses files they already send us today)

7. Risks & Mitigations

#	Risk	Likelihood	Impact	Mitigation
1	PG partner changes file format (column rename/drop)	Medium	High	Per-adapter column mapping; automated schema-change alert; ops can patch in the dashboard without code release for simple renames
2	Auto-match rate too low at launch	Medium	High	Pilot phase requires $\geq 80\%$ before rollout; rule sheet is plain-English Markdown that ops can tune
3	Finance team resists workflow change	Low	Medium	Co-design dashboard with them in Phase 2; keep Excel export

#	Risk	Likelihood	Impact	Mitigation
				option so they retain familiar artifact
4	Edge-case logic misses an SOP scenario	Medium	Medium	Build a labelled eval suite in Phase 2; finance team validates 1 full week before cutover; weekly drift checks
5	Bank Statement format varies day-to-day	Low	High	Existing adapter handles multiple formats already; statement is small (~15 rows) so manual fallback is trivial
6	Operator uploads wrong file to wrong slot	Low	Medium	Filename/content sanity checks at upload time; preview before reco runs
7	Dependency on a single backend engineer	Medium	Medium	Code is heavily commented; covered by docs/RECONCILIATION.md; backup engineer identified
8	Production database migration during rollout	Low	High	All schema changes are idempotent / additive; backwards-compatible with existing 5 configs
9	NPCI RRN collisions / multi-py_id per RRN — same UTR can map to two different transactions in our DB, or across PG partners	Medium	High (settling wrong row = real money error)	Composite join key for UPI: match on (UTR, amount, payer_vpa) — three signals together. Secondary cross-check against internal DB by py_id. Anything ambiguous → exception report, not auto-settled.
10	CSV leading-zero corruption — RRN like 064997745035 silently becomes 64997745035	High	High	All identifier columns parsed as STRINGS, never numbers. Adapter-level zero-padding when length is short. Upload-time detection scan flags suspect columns with a yellow banner.

#	Risk	Likelihood	Impact	Mitigation
	when touched by Excel or naive parsers; downstream joins fail			Every padded value is audit-logged.
11	Scale beyond v1 capacity (50k rows/source) — production volumes can reach lakhs (100k+) per day	Medium	High	v1 benchmarked at 50k rows / ≤ 5 min wall-clock. v2 scale plan documented: indexed match graph, batched DB writes, retention policy at 18 months. No LLM in critical path → scale bottleneck is pure CPU, not API cost.
12	LLM non-determinism in audit-critical decisions	Medium	Critical	Principle 1: LLM removed from the decision path entirely for settlement. Same input → byte-identical output, reproducible. LLM constrained to onboarding suggestions, exception narration text, and pattern alerts — all read-only and audit-logged.

8. Success Metrics

8.1 Pilot (YES Bank UPI) — go/no-go criteria for next PG

-  **Auto-match rate ≥ 80%** (rows the system fully resolves without human review)
-  **Zero false-settle** during 5-day side-by-side test (we never settle a row finance team flagged for refund/ignore)
-  **Zero missed refunds** (every Late-Authorized & Timeout row finance team would refund is also flagged by the system)
-  **Daily reco time per PG: 30+ min → ≤ 20 min**

-  **Finance team signs off** on dashboard ergonomics

8.2 Steady State (all 7 PGs)

-  **End-to-end audit trail** on every reconciliation decision (who, when, why)
-  **Daily reco time across all PGs: ~10 hrs → ≤ 2 hrs**
-  **New PG onboarding: 4 weeks → 1 week** (adapter + config + rules)
-  **Settlement error rate** (settled wrong / missed refund) **reduced ≥ 70%** vs. 12-month baseline

8.3 Stretch (post-rollout)

- Late-arriving file alerts surfaced proactively
- Pattern detection (e.g. "5 identical-amount Timeouts in 10 minutes — possible outage")
- Direct API intake (no manual upload)

9. Cost / Benefit Summary

Item	Today (annualised)	After (annualised)	Saving
Finance team daily reco effort	~10 person-hrs/day × 250 working days = 2,500 hrs/yr	~2 person-hrs/day = 500 hrs/yr	~2,000 hrs/yr
Cost of one missed refund / wrong settle	Variable; can be 4-figure ₹ per incident plus reputation	Detected automatically	Risk reduction
New PG onboarding analyst time	~4 weeks per new partner	Adapter + rules in ~1 week	~3 weeks per new partner
Audit / compliance support time	Spreadsheet archaeology when questioned	Query against persistent log	Faster compliance response

Net engineering cost: ~8 weeks of one backend engineer to reach YES Bank live + dashboard; ~12 additional weeks (part-time) for full 7-PG rollout. No incremental infrastructure spend.

10. Decision Required

We need sign-off on:

1. **Pilot PG choice:** YES Bank UPI (recommended — cleanest SOP, single mode, sample data already collected and verified)
 2. **Phased delivery model:** Phase 0 starts on approval; Phase 1 begins immediately after Discovery sign-off
 3. **Finance team time commitment:** 1 ops analyst available as design partner during Phase 2 (~10 hrs/week for 2 weeks); weekly check-ins thereafter
 4. **Success metric values** (auto-match threshold, time saving target) — proposed values listed in §8.1
-

11. Open Questions for Reviewer

These are explicit places I'd like the manager / Sr PM to weigh in before we lock the plan:

1. **PG sequencing after YES & NPST** — do we go by volume (which 5 of Paynext/Atom/TPSL/HDFC/RBL is highest?) or by which partner gives us the most pain today?
2. **Mode handling order** — UPI everywhere first, then revisit CC/DC/NB? Or finish all modes for YES before moving to the next PG?
3. **Refunds workflow** — should this be a follow-up phase to settlement (Q3), or is there urgency to start it in parallel?
4. **File ingestion automation** — is SMTP intake higher priority than rolling out more PGs after the pilot? (My recommendation: no — finish PG breadth first; automate intake in Q3.)
5. **Retention** — how long do we keep raw uploaded files + masked copies? Today: indefinite. Compliance team input needed.
6. **Ownership of per-PG rules** — once live, who maintains the disposition rule files ([disposition/settlement-yes-pg.ts](#)) and per-leg markdown rule sheets?

Finance ops? Engineering? Both with review?

7. **Cutover model** — parallel running (finance does manual + system runs) for how many days before we switch off Excel? Proposed: 5 days.
 8. **LLM scope** — Section 4.5 defines three narrow use cases (onboarding mapping suggestions, exception narration text, pattern alerts). Are we comfortable with even these? Would the team prefer a stricter "zero LLM in the reconciliation product, full stop" posture? Easy to do — affects only the optional decoration on the exception report.
 9. **Internal `py_id` echo upstream** — the composite-key approach (UTR + amount + payer VPA) works on raw files as they come today. A cleaner future would be asking each PG to echo our `py_id` back in their MIS so we can join directly. Worth opening that conversation with PG partner managers now, or wait until v1 ships? Doesn't block v1.
 10. **Confidence floor on auto-settle** — Should we expose a per-PG knob: "only auto-settle when X of N legs matched cleanly"? Or accept any pure-rule match as confident enough? Today's design assumes the latter.
-

12. Appendices

A. SOPs reviewed

- YES Bank UPI Transaction Settlement Process (from Finance Operations)
- NPST Settlement Process (from Finance Operations)

B. Sample data verified

- One real day of YES Bank reconciliation files (4 sheets in one workbook): MIS (999 rows), Statement (15 rows), Consolidated (3 rows), PG Incoming (15 rows). PII-masked locally; engineering has working sample for development & testing.

C. Engineering details (for technical reviewers)

- Built on Mastra workflows framework (already in production for other AI agents)
- Adapter pattern in `src/mastra/reconciliation/adapters/`

- Config-driven matching strategies: `exact`, `amount_tolerance`, `sum_then_match`, new `exclude_if_present` (anti-join), new composite-key form (`{ composite: ['utr', 'amountPaise', 'payerVpa'] }`)
- All identifier columns parsed as strings end-to-end; leading-zero preservation handled in shared `_csv-utils.ts` helper
- Per-PG disposition rules in TypeScript (`disposition/<configId>.ts`) — typed deterministic functions encoding SOP scenarios. Per-leg markdown rule sheets in `rules/<configId>/leg-N.md` document operator-facing intent
- OpenArc dashboard scope reduced: runs list + file-readiness widget + download buttons. No virtualised decisions table, no suspend/resume review banner (those are kept in the codebase but gated off for settlement)
- Report pack generation in `reports/report-pack-builder.ts` — streaming CSV writes; XLSX only for the exception report
- LLM components (`fuzzyMatchAgent`, `dispositionAgent`) stay in the codebase but gated off by per-config `llm: 'off'` flag for settlement workflows
- v1 capacity: 50k rows/source, ≤ 5 min wall-clock, fixture in `evals/fixtures/scale-50k/`
- Industry alignment validated against Modern Treasury (3-way matching + double-entry ledger), HighRadius (matching engine + AI auto-clear), Tipalti (real-time payout reconciliation), **EndClose** (YC-backed API-first reco platform — their AI is also constrained to rule-authoring, not match execution; same boundary as ours). Our deterministic-first stance is more conservative than HighRadius's 90% AI-cleared target — intentional, given audit expectations
- **Borrowed from EndClose for v1:** (1) field transforms at match time (`digits_only`, `lowercase`, etc.) to defeat Excel data drift, (2) unified tolerance primitive spanning amount + date in one clause, (3) rule provenance tracking (`source: user|ai|default`, `status: proposed|active`), (4) per-record SLA field on `ReconcileConfig` (data-model only in v1)
- **Deferred to v2** (designed-in but not built): Property Definitions (schema-less per-source fields), JSON filter DSL, webhook events (`record.reconciled`, `record.overdue`), status-as-derived state, OpenAPI auto-generation
- Full implementation plan: `~/ .claude/plans/bright-sauteeing-sonnet.md`

Reviewer comments welcome inline. Once approved, Phase 0 begins.